Clientes y servidores
Ejemplo simple en JavaScript y Express

En clase utilizamos VSCode para iniciar desde cero una aplicación muy simple utilizando Express como servidor JavaScript. Dado que la idea es familiarizarnos con el escenario general, usamos Copilot libremente.

1. En una carpeta vacía, ejecutamos **npm init.** Nos pedirá información, aceptemos todas las sugerencias. Luego podemos verificar que ha creado el archivo `package.json`
2. Instalemos Express ejecutando **npm install express**
3. Observemos que ahora tenemos una carpeta adicional `node_modules`.
4. En el `package.json` está indicado el archivo principal, que por defecto es `index.js`. Debemos crear ese archivo.
5. En clase utilicé comentarios en el código para sugerir lo que necesitaba a continuación. La ventaja es que trabajamos directamente en ese archivo, pero la desventaja es que la IA no tiene una visión global del proyecto y solo se limita a agregar código y algo puede fallar en el camino si es necesario importar librerías o algo similar (como efectivamente falló en clase). Para evitar eso podemos ejecutar directamente los siguientes prompts en Copilot <u>en modo agente</u>, lo que le permitirá hacer cambios más generales. Los prompts no tienen que ser exactamente iguales a los siguientes, pero sirven como referencia:
   i. **Prompt 1:** *Quiero una aplicación Express en el archivo index.js con un endpoint que muestre un cartel "Hola Mundo" .*
      - Observen el código generado. Puede variar con el código generado por otros compañeros u otras ocurrencias del mismo ejemplo. Algunas cosas son frecuentes, en otras puede elegir cosas diferentes ¿qué port ha usado? ¿Cómo indica que el servidor ya está activado? ¿ha generado alguna URL particular, como "`/saludo`" o usa directamente el home "`/`"?

   ii. **Prompt 2:** *Quiero un endpoint que dado un nombre, salude a la persona con ese nombre.*
      - Aquí hay también variaciones ¿Qué URL indica usar? A veces genera "`/saludo/:nombre`", a veces "`/saludo?nombre=XYZ`".

   iii. **Prompt 3**: *Quiero un endpoint que me genere los datos de un empleado ficticio, utilizando faker.*
      - Esto requiere importar la librería faker y debería hacerse cargo de eso. Verifiquen el código y vean cómo genera los datos falsos. Nunca le pedimos que sea en formato JSON, pero seguramente decidirá ofrecerlos con ese formato.

   iv. **Prompt 4**: *Quiero un endpoint que dado un número X, me devuelva una lista de X empleados generados con faker.*
      - De nuevo, verifiquen el código y cómo genera la lista de empleados.

   v. **Prompt 5:** *Quiero una pagina HTML, estilizada apropiadamente, que use el endpoint anterior para mostrarme los datos de los empleados en una tabla.*
      - Aquí generará un archivo nuevo. Además hará un cambio en el index.js para poder acceder a ese HTML. Observen el cambio propuesto.
      - Observen cómo la página HTML accede al endpoint de la lista de empleados.
      - Observen en las Herramientas de Desarrollador del navegador, cómo interactúa la página web con el servidor que le provee la lista de empleados.